

Paralelização de Aplicação com a Biblioteca MPI

Artur Marcon, Giovana Raupp e Vitoria Gonzalez

1 Introdução

Este trabalho implementa uma versão paralela do problema de geração de uma animação de *zoom* (dive) no conjunto de Julia, utilizando MPI no modelo coordenador/trabalhador. A unidade de trabalho é um *frame* completo de 1920×1080 pixels com `MAX_ITER` = 2000 iterações, e o saco de trabalho contém N frames com fator de zoom progressivo $0,94^i$. Esse fator torna o custo computacional heterogêneo entre os frames (frames mais profundos atingem `MAX_ITER` com mais frequência), o que justifica o uso de uma fila dinâmica para balanceamento de carga.

2 Metodologia

Comunicação MPI. A iniciativa é dos trabalhadores, conforme o enunciado: cada worker envia um `TAG_READY` inicial ao mestre indicando que está disponível, e o mestre é puramente reativo (sem bootstrap). No loop principal o mestre usa `MPI_Probe` para inspecionar a tag/origem da próxima mensagem antes de chamar `MPI_Recv`, necessário porque ele recebe dois tipos de mensagens de tamanhos muito diferentes: 4 bytes para um `TAG_READY` e ~6MB para o buffer RGB de um resultado. Após processar a mensagem, o mestre responde com `TAG_WORK` (próximo *frame id*) ou `TAG_DIE` (fim do trabalho). O resultado de cada frame carrega o próprio *frame id* como tag, dispensando mensagem extra de metadados e funcionando implicitamente como o próximo pedido daquele worker. As estatísticas por trabalhador (frames processados e tempo de cálculo) são coletadas após o loop principal via `MPI_Gather`, isolando-as do tráfego do despachante. Uma variante *master-driven* (sem `TAG_READY`, com bootstrap no mestre) foi também medida e apresentou diferenças < 2% em todas as configurações, confirmando que o custo da mensagem adicional é desprezível.

Saco de trabalho. O modelo canônico mantém uma matriz de vetores em memória que serve simultaneamente como buffer de envio e de recebimento, com `MPI_Probe` + `MPI_Recv` direto na posição final. Não usamos essa estrutura literalmente: o mestre não envia vetores de entrada (apenas o *frame id* de 4 bytes), e a saída natural de cada frame é um arquivo PPM em disco. A ordem original do saco é preservada pelo nome do arquivo (`frame_NNNN.ppm`), evitando os ~1,86GB de RAM que uma matriz materializada de 300 frames a 1920×1080 exigiria no rank 0.

Ambiente experimental. Cluster *atlantica* (LAD/PU-CRS), 2 nós exclusivos alocados via `salloc -exclusive -N 2` dentro da mesma sessão (Ubuntu 20.04, gcc 9.4.0, OpenMPI 4.1.1; cada nó com 8 cores físicos / 16 threads HT). Tanto o baseline sequencial quanto a versão paralela executam em nós de cálculo da mesma alocação (o sequencial via `srun -n 1 -N 1`, garantindo hardware idêntico). Ambos compilados com `-O3 -Wall`. Cada configuração executada **3 vezes**; reporta-se a mediana. I/O de PPM desligado durante as medições para isolar custo de cálculo.

Métricas. Sendo W o número de *trabalhadores* ($W = np - 1$): $S_f(W) = \frac{T_{seq}}{T(W)}$, $E_f(W) = \frac{S_f(W)}{W}$
 $S_w(W) = \frac{T(W=1)}{T(W)}$, $E_w(W) = S_w(W)$ Para a escalabilidade fraca, fixa-se 10 frames por trabalhador, conforme orientação do enunciado de *contar apenas o trabalhador* para o cálculo. O *desbalanceamento* é definido como $(\max - \min) / \max$ entre os tempos de cálculo dos trabalhadores.

3 Resultados

A Tabela 1 apresenta a escalabilidade forte (60 frames) e a Tabela 2 a fraca. Os gráficos correspondentes estão na Figura 1.

Escalabilidade forte. O speedup cresce de 1,0 ($W=1$, esperado) até 21,4× ($W=31$). A eficiência mantém-se acima de 88% até $W=15$ (eficiência 0,88 em $W=15$, 0,92 em $W=7$, 0,98 em $W=3$) e cai para 69% em $W=31$. Os fatores limitantes observados foram: (i) o *rank 0* é serializado, recebendo um buffer RGB de ~6 MB por frame, processando o resultado, e respondendo com o próximo *frame id*; (ii) com apenas 60 frames distribuídos entre 31 trabalhadores, a granularidade torna-se grosseira: em média 1,9 frames por worker, ou seja, parte processa 2 frames e o restante apenas 1, gerando o salto de desbalanceamento de 15% ($W=15$) para 54% ($W=31$).

Escalabilidade fraca. A eficiência fraca cai de 1,000 ($W=1$) para 0,24 ($W=31$). Essa degradação não é apenas overhead: ao escalar o número total de frames junto com o número de trabalhadores ($N = 10W$), aumenta-se também a *média* do índice de cada frame — e portanto a profundidade do zoom — de modo que a carga média por frame cresce em vez de permanecer constante. Em $W=31$ os frames atingem o índice 309 (zoom $\approx 0,94^{309} \approx 6 \times 10^{-9}$), com tempo por frame significativamente superior ao da configuração baseline ($W=1$, frames 0 a 9).

Balanceamento de carga. O modelo *coordenador/trabalhador*, com iniciativa dos trabalhadores, entrega balanceamento dinâmico naturalmente: workers que terminam antes recebem mais trabalho. O desbalanceamento medido cresce com W (2% em $W=3$, 12% em $W=7$, 15% em $W=15$, 54% em $W=31$), mas o salto em $W=31$ é *estrutural* (poucos frames por worker) e não um problema do algoritmo. Aumentar N recuperaria o balanceamento.

4 Conclusão

A versão MPI mestre/trabalhador atinge speedup forte de 21,4× com 31 trabalhadores (eficiência 69%), validando o modelo para a carga heterogênea da animação. Até $W=15$ a eficiência permanece $\geq 88\%$. As principais limitações observadas são: (a) serialização do rank 0 (recebe ~6 MB por frame), e (b) granularidade insuficiente em $W=31$ com apenas 60 frames. Trabalhos futuros incluem escrita de PPM delegada ao próprio trabalhador (reduz tráfego) e sobreposição de comunicação/computação com `MPI_Isend/MPI_Irecv`.

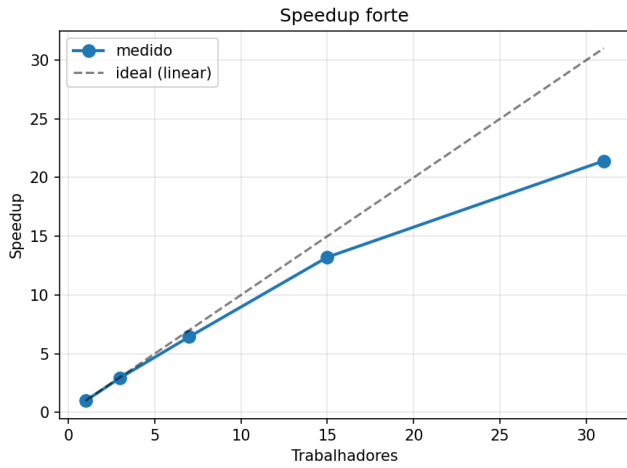
Tabelas e Gráficos

Tabela 1: Escalabilidade forte (60 frames, mediana de 3 reps). $T_{seq} = 163,87$ s.

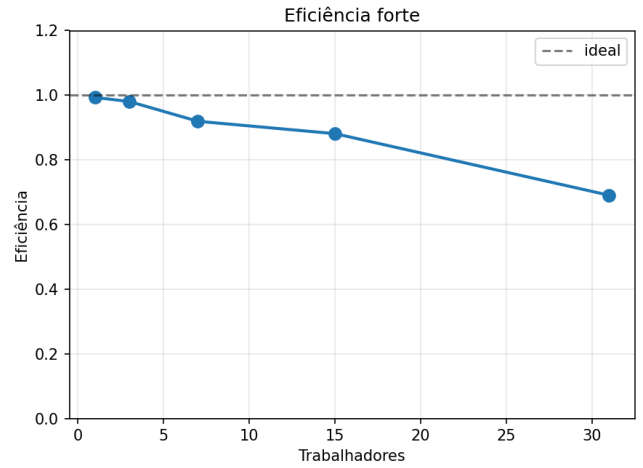
np	W	T (s)	S	E	Desbal. (%)
2	1	164,96	0,993	0,993	0,00
4	3	55,70	2,942	0,981	2,48
8	7	25,46	6,437	0,920	12,31
16	15	12,40	13,220	0,881	15,25
32	31	7,65	21,413	0,691	54,29

Tabela 2: Escalabilidade fraca (10 frames/worker, mediana de 3 reps). $T(W=1) = 17,97$ s.

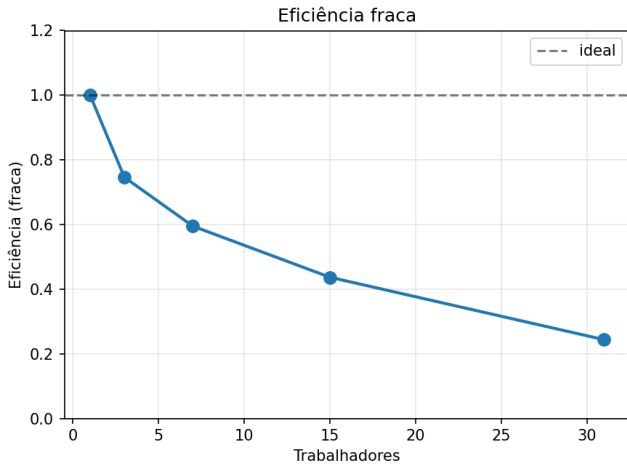
np	W	frames	T (s)	S	E
2	1	10	17,97	1,000	1,000
4	3	30	24,08	0,746	0,746
8	7	70	30,21	0,595	0,595
16	15	150	41,13	0,437	0,437
32	31	310	73,71	0,244	0,244



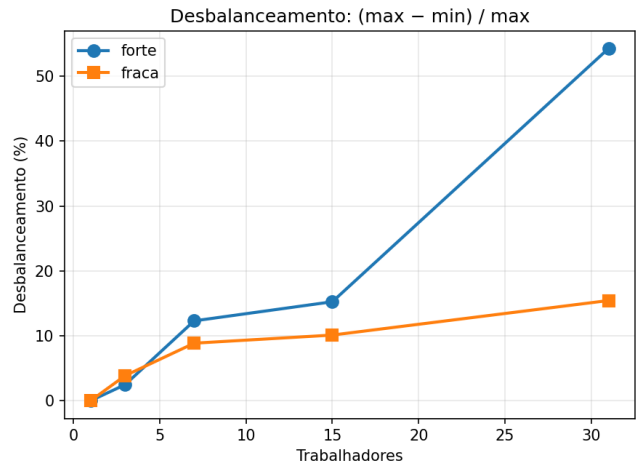
(a) Speedup forte



(b) Eficiência forte



(c) Eficiência fraca



(d) Desbalanceamento

Figura 1: Curvas de escalabilidade e balanceamento de carga.

Código-fonte

julia_dive.c — versão paralela MPI (mestre/trabalhador)

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <math.h>
5
6 #define WIDTH 1920
7 #define HEIGHT 1080
8 #define MAX_ITER 2000
9 #define DEFAULT_TOTAL_FRAMES 300
10
11 // MPI Message Tags
12 #define TAG_WORK 1 // master -> worker: here's a frame_id to compute
13 #define TAG_DIE 2 // master -> worker: no more work, terminate
14 #define TAG_READY 99999 // worker -> master: initial "I want work" request.
15 // Result messages reuse the frame_id as their tag, so
16 // TAG_READY must be > any valid frame_id to avoid
17 // collision (frame_id=3 vs TAG_READY=3 MPI_Probe
18 // can't tell them apart and Recv truncates).
19
20 // Julia set constant configuration
21 const double C_REAL = -0.7;
22 const double C_IMAG = 0.27015;
23
24 // Define 3 cool colors matching the first program: Night Blue -> Electric Cyan -> Deep Purple
25 const int COLOR1_R = 10; const int COLOR1_G = 15; const int COLOR1_B = 45; // Night Blue
26 const int COLOR2_R = 0; const int COLOR2_G = 225; const int COLOR2_B = 235; // Electric Cyan
27 const int COLOR3_R = 140; const int COLOR3_G = 25; const int COLOR3_B = 230; // Deep Purple
28
29 // Computes a single frame's pixel data into a raw RGB buffer
30 void compute_julia_frame(int frame_id, double center_real, double center_imag, unsigned char *buffer) {
31     double zoom = pow(0.94, frame_id);
32
33     double base_width = 3.0;
34     double current_width = base_width * zoom;
35     double current_height = current_width * ((double)HEIGHT / (double)WIDTH);
36
37     for (int y = 0; y < HEIGHT; y++) {
38         for (int x = 0; x < WIDTH; x++) {
39             double z_real = center_real + (x - WIDTH / 2.0) * (current_width / WIDTH);
40             double z_imag = center_imag + (y - HEIGHT / 2.0) * (current_height / HEIGHT);
41
42             int iter = 0;
43             double z_real_sq = z_real * z_real;
44             double z_imag_sq = z_imag * z_imag;
45
46             while (z_real_sq + z_imag_sq <= 4.0 && iter < MAX_ITER) {
47                 double temp = z_real_sq - z_imag_sq + C_REAL;
48                 z_imag = 2.0 * z_real * z_imag + C_IMAG;
49                 z_real = temp;
50
51                 z_real_sq = z_real * z_real;
52                 z_imag_sq = z_imag * z_imag;
53                 iter++;
54             }
55
56             size_t pixel_idx = (size_t)(y * WIDTH + x) * 3;
57
58             if (iter == MAX_ITER) {
59                 // Fractal interior (Black)
60                 buffer[pixel_idx] = 0;
61                 buffer[pixel_idx + 1] = 0;
62                 buffer[pixel_idx + 2] = 0;
63             } else {
64                 // Normalized iteration value for smooth 3-color transition
65                 double mu = (double)iter / MAX_ITER;
66
67                 if (mu < 0.5) {
68                     double t = mu * 2.0; // Scale to [0, 1]
69                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR1_R + t * COLOR2_R);
70                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR1_G + t * COLOR2_G);
71                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR1_B + t * COLOR2_B);
72                 } else {
73                     double t = (mu - 0.5) * 2.0; // Scale to [0, 1]
74                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR2_R + t * COLOR3_R);
75                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR2_G + t * COLOR3_G);
76                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR2_B + t * COLOR3_B);
77                 }
78             }
79         }
80     }
81 }
82
83 void save_ppm(int frame_id, unsigned char *buffer) {
84     char filename[64];
85     sprintf(filename, "frame_%04d.ppm", frame_id);
86
87     FILE *fp = fopen(filename, "wb");
88     if (!fp) {
89         fprintf(stderr, "Error opening file %s for writing.\n", filename);
90     }
91 }
```

```

90     return;
91 }
92
93 fprintf(fp, "P6\n%d %d\n255\n", WIDTH, HEIGHT);
94 fwrite(buffer, sizeof(unsigned char), (size_t)WIDTH * HEIGHT * 3, fp);
95 fclose(fp);
96 }
97
98 int main(int argc, char *argv[]) {
99     int rank, size;
100
101     MPI_Init(&argc, &argv);
102     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
103     MPI_Comm_size(MPI_COMM_WORLD, &size);
104
105     double target_real = -0.1;
106     double target_imag = -0.1;
107     int total_frames = DEFAULT_TOTAL_FRAMES;
108     int save_files = 1;
109
110     if (argc >= 3) {
111         target_real = atof(argv[1]);
112         target_imag = atof(argv[2]);
113     }
114     if (argc >= 4) {
115         total_frames = atoi(argv[3]);
116     }
117     if (argc >= 5) {
118         save_files = atoi(argv[4]);
119     }
120
121     if (size < 2) {
122         if (rank == 0) {
123             fprintf(stderr, "Error: This Master-Worker model requires at least 2 MPI processes.\n");
124             fprintf(stderr, "Usage: %s [target_real target_imag [total_frames [save_files]]]\n", argv[0]);
125         }
126         MPI_Finalize();
127         return 1;
128     }
129
130     size_t frame_buffer_size = (size_t)WIDTH * HEIGHT * 3 * sizeof(unsigned char);
131
132     if (rank == 0) {
133         // ===== MASTER RANK =====
134         printf("[Master] Rendering %d frames into (%f, %f), workers=%d, save_files=%d\n",
135             total_frames, target_real, target_imag, size - 1, save_files);
136         double start_time = MPI_Wtime();
137         double io_time = 0.0;
138
139         int next_frame = 0;
140         int active_workers = size - 1;
141
142         // Purely reactive: no bootstrap. The initiative is on the workers --
143         // they either ping us with TAG_READY (first contact) or send back a
144         // completed frame (tag = frame_id, acting as an implicit next request).
145         // MPI_Probe lets us peek at the tag/source so we know which kind of
146         // message it is BEFORE issuing the matching MPI_Recv (small int vs
147         // 6 MB buffer).
148         unsigned char *recv_buffer = (unsigned char *)malloc(frame_buffer_size);
149         while (active_workers > 0) {
150             MPI_Status status;
151             MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
152
153             int worker_id = status.MPI_SOURCE;
154             int tag = status.MPI_TAG;
155
156             if (tag == TAG_READY) {
157                 int dummy;
158                 MPI_Recv(&dummy, 1, MPI_INT, worker_id, TAG_READY,
159                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
160             } else {
161                 int completed_frame_id = tag;
162                 MPI_Recv(recv_buffer, (int)frame_buffer_size, MPI_BYTE,
163                     worker_id, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
164                 if (save_files) {
165                     double io0 = MPI_Wtime();
166                     save_ppm(completed_frame_id, recv_buffer);
167                     io_time += MPI_Wtime() - io0;
168                 }
169             }
170
171             if (next_frame < total_frames) {
172                 MPI_Send(&next_frame, 1, MPI_INT, worker_id, TAG_WORK, MPI_COMM_WORLD);
173                 next_frame++;
174             } else {
175                 MPI_Send(&next_frame, 1, MPI_INT, worker_id, TAG_DIE, MPI_COMM_WORLD);
176                 active_workers--;
177             }
178         }
179
180         free(recv_buffer);
181         double end_time = MPI_Wtime();
182         double total_time = end_time - start_time;
183     }

```

```

184 // Collect per-worker stats for load-balance analysis using a collective
185 // gather. Done AFTER the dispatch loop so the main-loop MPI_Recv (which
186 // uses MPI_ANY_TAG) cannot accidentally consume stats messages.
187 double local_stats[2] = {0.0, 0.0};
188 double *all_stats = (double *)malloc(2 * size * sizeof(double));
189 MPI_Gather(local_stats, 2, MPI_DOUBLE, all_stats, 2, MPI_DOUBLE, 0, MPI_COMM_WORLD);
190
191 printf("[Master] total=%.4fs io=%.4fs comm+wait=%.4fs\n",
192        total_time, io_time, total_time - io_time);
193 printf("[Master] worker_id,frames_processed,compute_time_s\n");
194
195 int total_frames_check = 0;
196 double max_compute = 0.0, min_compute = 1e18, sum_compute = 0.0;
197 for (int worker = 1; worker < size; worker++) {
198     int wframes = (int)all_stats[2 * worker];
199     double wtime = all_stats[2 * worker + 1];
200     total_frames_check += wframes;
201     if (wtime > max_compute) max_compute = wtime;
202     if (wtime < min_compute) min_compute = wtime;
203     sum_compute += wtime;
204     printf("[Master] %d,%d,%.4f\n", worker, wframes, wtime);
205 }
206 double avg_compute = sum_compute / (size - 1);
207 double imbalance = (max_compute - min_compute) / (max_compute > 0 ? max_compute : 1.0);
208 printf("[Master] frames_total=%d avg_compute=%.4fs max=%.4f min=%.4f imbalance=%.2f%%\n",
209        total_frames_check, avg_compute, max_compute, min_compute, imbalance * 100.0);
210
211 // Machine-readable summary line (consumed by run_experiments.sh):
212 // CSV,kind,np,workers,total_frames,total_s,io_s,compute_max_s,compute_min_s,imbalance_pct
213 printf("CSV,par,%d,%d,%d,%.6f,%.6f,%.6f,%.6f,%.4f\n",
214        size, size - 1, total_frames,
215        total_time, io_time, max_compute, min_compute, imbalance * 100.0);
216 free(all_stats);
217
218 } else {
219     // ===== WORKER RANKS =====
220     unsigned char *local_buffer = (unsigned char *)malloc(frame_buffer_size);
221     int frames_processed = 0;
222     double compute_time = 0.0;
223
224     // Worker takes the initiative (per enunciado): announce readiness
225     // before any computation. After this, sending a result implicitly
226     // serves as the next "ready" signal.
227     int ready_payload = rank;
228     MPI_Send(&ready_payload, 1, MPI_INT, 0, TAG_READY, MPI_COMM_WORLD);
229
230     while (1) {
231         int frame_to_build;
232         MPI_Status status;
233
234         MPI_Recv(&frame_to_build, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
235
236         if (status.MPI_TAG == TAG_DIE) {
237             break;
238         }
239
240         double c0 = MPI_Wtime();
241         compute_julia_frame(frame_to_build, target_real, target_imag, local_buffer);
242         compute_time += MPI_Wtime() - c0;
243         frames_processed++;
244
245         MPI_Send(local_buffer, (int)frame_buffer_size, MPI_BYTE, 0, frame_to_build, MPI_COMM_WORLD);
246     }
247
248     double local_stats[2] = { (double)frames_processed, compute_time };
249     MPI_Gather(local_stats, 2, MPI_DOUBLE, NULL, 0, MPI_DOUBLE, 0, MPI_COMM_WORLD);
250
251     free(local_buffer);
252 }
253
254 MPI_Finalize();
255 return 0;
256 }

```

julia_dive_seq.c — versão sequencial (baseline)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <time.h>
5
6 #define WIDTH 1920
7 #define HEIGHT 1080
8 #define MAX_ITER 2000
9 #define DEFAULT_TOTAL_FRAMES 300
10
11 const double C_REAL = -0.7;
12 const double C_IMAG = 0.27015;
13
14 const int COLOR1_R = 10;    const int COLOR1_G = 15;    const int COLOR1_B = 45;
15 const int COLOR2_R = 0;    const int COLOR2_G = 225;    const int COLOR2_B = 235;

```

```

16 const int COLOR3_R = 140;  const int COLOR3_G = 25;   const int COLOR3_B = 230;
17
18 void compute_julia_frame(int frame_id, double center_real, double center_imag, unsigned char *buffer) {
19     double zoom = pow(0.94, frame_id);
20
21     double base_width = 3.0;
22     double current_width = base_width * zoom;
23     double current_height = current_width * ((double)HEIGHT / (double)WIDTH);
24
25     for (int y = 0; y < HEIGHT; y++) {
26         for (int x = 0; x < WIDTH; x++) {
27             double z_real = center_real + (x - WIDTH / 2.0) * (current_width / WIDTH);
28             double z_imag = center_imag + (y - HEIGHT / 2.0) * (current_height / HEIGHT);
29
30             int iter = 0;
31             double z_real_sq = z_real * z_real;
32             double z_imag_sq = z_imag * z_imag;
33
34             while (z_real_sq + z_imag_sq <= 4.0 && iter < MAX_ITER) {
35                 double temp = z_real_sq - z_imag_sq + C_REAL;
36                 z_imag = 2.0 * z_real * z_imag + C_IMAG;
37                 z_real = temp;
38
39                 z_real_sq = z_real * z_real;
40                 z_imag_sq = z_imag * z_imag;
41                 iter++;
42             }
43
44             size_t pixel_idx = (size_t)(y * WIDTH + x) * 3;
45
46             if (iter == MAX_ITER) {
47                 buffer[pixel_idx] = 0;
48                 buffer[pixel_idx + 1] = 0;
49                 buffer[pixel_idx + 2] = 0;
50             } else {
51                 double mu = (double)iter / MAX_ITER;
52
53                 if (mu < 0.5) {
54                     double t = mu * 2.0;
55                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR1_R + t * COLOR2_R);
56                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR1_G + t * COLOR2_G);
57                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR1_B + t * COLOR2_B);
58                 } else {
59                     double t = (mu - 0.5) * 2.0;
60                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR2_R + t * COLOR3_R);
61                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR2_G + t * COLOR3_G);
62                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR2_B + t * COLOR3_B);
63                 }
64             }
65         }
66     }
67 }
68
69 void save_ppm(int frame_id, unsigned char *buffer) {
70     char filename[64];
71     sprintf(filename, "frame_%04d.ppm", frame_id);
72
73     FILE *fp = fopen(filename, "wb");
74     if (!fp) {
75         fprintf(stderr, "Error opening file %s for writing.\n", filename);
76         return;
77     }
78
79     fprintf(fp, "P6\n%d %d\n255\n", WIDTH, HEIGHT);
80     fwrite(buffer, sizeof(unsigned char), (size_t)WIDTH * HEIGHT * 3, fp);
81     fclose(fp);
82 }
83
84 static double now_seconds(void) {
85     struct timespec ts;
86     clock_gettime(CLOCK_MONOTONIC, &ts);
87     return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
88 }
89
90 int main(int argc, char *argv[]) {
91     double target_real = -0.1;
92     double target_imag = -0.1;
93     int total_frames = DEFAULT_TOTAL_FRAMES;
94     int save_files = 1;
95
96     if (argc >= 3) {
97         target_real = atof(argv[1]);
98         target_imag = atof(argv[2]);
99     }
100     if (argc >= 4) {
101         total_frames = atoi(argv[3]);
102     }
103     if (argc >= 5) {
104         save_files = atoi(argv[4]);
105     }
106
107     size_t frame_buffer_size = (size_t)WIDTH * HEIGHT * 3 * sizeof(unsigned char);
108     unsigned char *buffer = (unsigned char *)malloc(frame_buffer_size);
109     if (!buffer) {

```

```

110         fprintf(stderr, "Error: failed to allocate frame buffer.\n");
111         return 1;
112     }
113
114     printf("[Seq] Rendering %d frames into (%f, %f), save_files=%d\n",
115           total_frames, target_real, target_imag, save_files);
116
117     double t_compute = 0.0;
118     double t_io = 0.0;
119     double t_start = now_seconds();
120
121     for (int frame_id = 0; frame_id < total_frames; frame_id++) {
122         double tc0 = now_seconds();
123         compute_julia_frame(frame_id, target_real, target_imag, buffer);
124         double tc1 = now_seconds();
125         t_compute += (tc1 - tc0);
126
127         if (save_files) {
128             double ti0 = now_seconds();
129             save_ppm(frame_id, buffer);
130             double ti1 = now_seconds();
131             t_io += (ti1 - ti0);
132         }
133     }
134
135     double t_end = now_seconds();
136     double t_total = t_end - t_start;
137
138     printf("[Seq] total=%.4fs compute=%.4fs io=%.4fs\n", t_total, t_compute, t_io);
139
140     // Machine-readable summary line (consumed by run_experiments.sh):
141     // CSV,kind,np,workers,total_frames,total_s,io_s,compute_max_s,compute_min_s,imbalance_pct
142     printf("CSV,seq,1,0,%d,%.6f,%.6f,%.6f,%.6f,0.0000\n",
143           total_frames, t_total, t_io, t_compute, t_compute);
144
145     free(buffer);
146     return 0;
147 }

```

run_experiments.sh — script de experimentos

```

1  #!/usr/bin/env bash
2  # Run strong and weak scaling experiments and emit a CSV ready for analysis.
3  #
4  # Compile first:
5  #   local: mpicc -O3 -Wall -o julia_dive julia_dive.c -lm
6  #           cc -O3 -Wall -o julia_dive_seq julia_dive_seq.c -lm
7  #   lad:   ladcomp -env mpicc -O3 -Wall julia_dive.c -o julia_dive -lm
8  #           gcc -O3 -Wall julia_dive_seq.c -o julia_dive_seq -lm
9  #   (-O3 is CRITICAL: without it, ladcomp/mpicc defaults to -O0 and the
10 #    parallel binary runs ~2.5x slower than the sequential, falsifying speedup.)
11 #
12 # Usage:
13 # ./run_experiments.sh local    # Mac/dev box validation (mpirun, smaller workload)
14 # ./run_experiments.sh lad      # PUCRS LAD atlantica (srun, full workload)
15 #
16 # IMPORTANT (lad mode): run inside a salloc allocation so all srun calls
17 # share the same exclusive nodes (consistent hardware across reps):
18 #   salloc --exclusive -N 2 -t 00:30:00
19 # ./run_experiments.sh lad
20 # exit
21 #
22 # Tunables (env vars): REPS, FRAMES_STRONG, FRAMES_PER_WORKER
23 # Output: results_<mode>.csv
24
25 set -euo pipefail
26
27 MODE="${1:-local}"
28 REPS="${REPS:-3}"
29
30 case "$MODE" in
31     local)
32         NP_LIST=(2 4 8)
33         FRAMES_STRONG="${FRAMES_STRONG:-60}"
34         FRAMES_PER_WORKER="${FRAMES_PER_WORKER:-10}"
35         LAUNCHER=(mpirun -np)
36         SEQ_PREFIX=()
37         ;;
38     lad)
39         # PUCRS LAD atlantica. Assumes you are running INSIDE a salloc allocation
40         # (see header comment)      node count, exclusivity and walltime come from
41         # the surrounding salloc, so each srun just specifies -n.
42         # 16 and 32 are the mandatory points; intermediates fill the speedup curve.
43         # SEQ_PREFIX forces the sequential baseline to run on a COMPUTE node too,
44         # otherwise it executes on the (faster) login node and falsifies T_seq.
45         NP_LIST=(2 4 8 16 32)
46         FRAMES_STRONG="${FRAMES_STRONG:-300}"
47         FRAMES_PER_WORKER="${FRAMES_PER_WORKER:-20}"
48         LAUNCHER=(srun -N 2 -n)      # force tasks spread across both nodes;
49                                     # without -N 2, srun packs onto one node and
50                                     # master+worker collide on hyperthreads.

```

```

51     SEQ_PREFIX=(srun -n 1 -N 1) # -N 1 silences "1 proc on 2 nodes" warning
52     ;;
53 *)
54     echo "usage: $0 [local|lad]" >&2
55     exit 1
56     ;;
57 esac
58
59 OUT="results_${MODE}.csv"
60 echo "experiment,rep,kind,np,workers,total_frames,total_s,io_s,compute_max_s,compute_min_s,imbalance_pct" > "$OUT"
61
62 run_seq() {
63     local exp="$1" rep="$2" frames="$3"
64     echo "[run] $exp rep=$rep seq frames=$frames"
65     local line
66     line=$(("${SEQ_PREFIX[@]}") ./julia_dive_seq -0.1 -0.1 "$frames" 0 | grep '^CSV,' | sed 's/^CSV,/' )
67     echo "$exp,$rep,$line" >> "$OUT"
68 }
69
70 run_par() {
71     local exp="$1" rep="$2" np="$3" frames="$4"
72     echo "[run] $exp rep=$rep np=$np frames=$frames"
73     local line
74     line=$(("${LAUNCHER[@]}") "$np" ./julia_dive -0.1 -0.1 "$frames" 0 | grep '^CSV,' | sed 's/^CSV,/' )
75     echo "$exp,$rep,$line" >> "$OUT"
76 }
77
78 echo "=== Strong scaling (frames=$FRAMES_STRONG) ==="
79 for rep in $(seq 1 "$REPS"); do
80     run_seq strong "$rep" "$FRAMES_STRONG"
81     for np in "${NP_LIST[@]}; do
82         run_par strong "$rep" "$np" "$FRAMES_STRONG"
83     done
84 done
85
86 echo "=== Weak scaling (~$FRAMES_PER_WORKER frames/worker) ==="
87 for rep in $(seq 1 "$REPS"); do
88     for np in "${NP_LIST[@]}; do
89         workers=$((np - 1))
90         frames=$((workers * FRAMES_PER_WORKER))
91         run_par weak "$rep" "$np" "$frames"
92     done
93 done
94
95 echo
96 echo "Done. Results in: $OUT"

```